Template Course

Week title

Jane Doe John Doe

June 2, 2026

Slide 1 Title

- Point 1
- Point 2

Images

Example img

Rule 2: Use Participatory Live Coding

> Actually type the code and narrate it as you teach. Have the participants follow along as well.

- Demonstrates your **best practices** for real workflows
- Making **mistakes** is a good thing — it shows you are human and how you **debug**
- Naturally **slows you down**, so you don't go too fast for students

Rooted in the *"I do, we do, you do"* method, pioneered for programming by [The Carpentries](). The `{pyodide}` cells on the next slides let students follow along live, in the browser.

Timbers, T. A., & Çetinkaya-Rundel, M. (2026). Ten simple rules for teaching data science. *PLoS Computational Biology*.
arXiv:2602.02874

Interactive Code

Exercise Start Over Run Code

```python

# square each number
for x in range(5):
  print(______)
```

[More documentation on Live Code]()

Solution Cells

Attach a hidden solution (and optional hint) to any `exercise`.

A *Show solution* button appears under the editor.

Exercise | Start Over | Show Hint | Run Code

```
# square each number
for x in range(5):
  print(______)
```

Public/student builds keep solutions; set `show-solutions: false` in the front matter to strip them.

Line Highlighting

```python
import numpy as np
import matplotlib.pyplot as plt

r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
ax.plot(theta, r)
ax.set_rticks([0.5, 1, 1.5, 2])
ax.grid(True)
plt.show()
```

Line Higlighting with animation

```python
import numpy as np
import matplotlib.pyplot as plt

r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
ax.plot(theta, r)
ax.set_rticks([0.5, 1, 1.5, 2])
ax.grid(True)
plt.show()
```

Multiple columns

Left column

Right column

Aside

- Green [1]
- Brown
- Purple

Some additional commentary of more peripheral interest.

1. A footnote

Multiple Choice (QCM)

The `qcm` filter renders a self-checking, auto-shuffled question. Mark answers `\+` (correct) / `\-` (wrong).

Valid Python container types? (1 point)

- [ ] set
- [ ] array (built-in)
- [ ] dict
- [ ] list

Verify

`solution="true"` adds a **Verify** button; `points="N"` shows in the heading.

Free-Text Answers

The `textanswer` filter renders a printable answer box for worksheets and PDF exports.

What does a list comprehension do? (3 points)

Answer...

`lines` sets the box height; `points` shows in the heading.

Recap Quizzes → Moodle

Each week can ship a `questions.md` file. Write questions once in plain markdown; export them to a Moodle XML question bank.

```
# Week 1 — Recap

**1. What does `len(x)` return for a list?**
- *A) The number of elements
- B) The sum of the elements
- C) The largest element
```

A leading `*` marks the correct answer. Inline `code`, fenced code blocks and `![images](pic.png)` are supported (images are embedded as base64).

Recap Quizzes → Moodle

Convert with the bundled script, then import the `.xml` via **Question bank** → **Import** → **Moodle XML**:

```
# Convert a single week
python extensions/questions_to_moodle.py Week1/questions.md

# Convert every Week*/questions.md at once
python extensions/questions_to_moodle.py --all

# Collect all XML in one folder
python extensions/questions_to_moodle.py --all --output-dir moodle_xml
```

Jupyter Notebook Flow

Weekly labs are in-browser Jupyter notebooks:

1. Open the link — runs in the browser, **no install**.
2. Built-in submission form (student e-mail).
3. Work saved **locally in the browser** — reopen to resume.
4. Submit through the embedded form for grading.

Reference solutions → PDF via `extensions/convert_solutions.py`.

Incremental Lists

Add `.incremental` to reveal list items one click at a time.

- First point appears
- Then the second
- Finally the third

Set `incremental: true` in the front matter to make it the default for every list.

Highlight the Current Item

Combine `.incremental` with `.highlight-last` to dim earlier points and keep focus on the one just revealed.

- Collect the data
- Clean and explore it
- Model and evaluate
- Communicate the result

Timeline

The `timeline` filter renders events along a line. Each `.event` has a `data-label`.

Week 1	Week 2	Week 3	...
Python & tooling	Collecting data	Visualisation	Group project

Arrows

The `arrow` shortcode draws curved SVG arrows to annotate slides.

look here!

Cause → Effect

Use `position="absolute"` with pixel coordinates to point anywhere on the slide:

Install with `quarto add EmilHvitfeldt/quarto-arrows`. Options include `curve`, `bend`, `dash`, `label`, and `fragment` for draw-on-click.

Iframe

Embed live web content (polls, dashboards, demos):

Participate in an event

Enter event code

e.g.: ABCDEF

Join event

Welcome back!

How would you like to log in?

Interactive iframes grab keyboard focus, so keep them on the **last** slide — otherwise arrow-key navigation can get trapped here.